



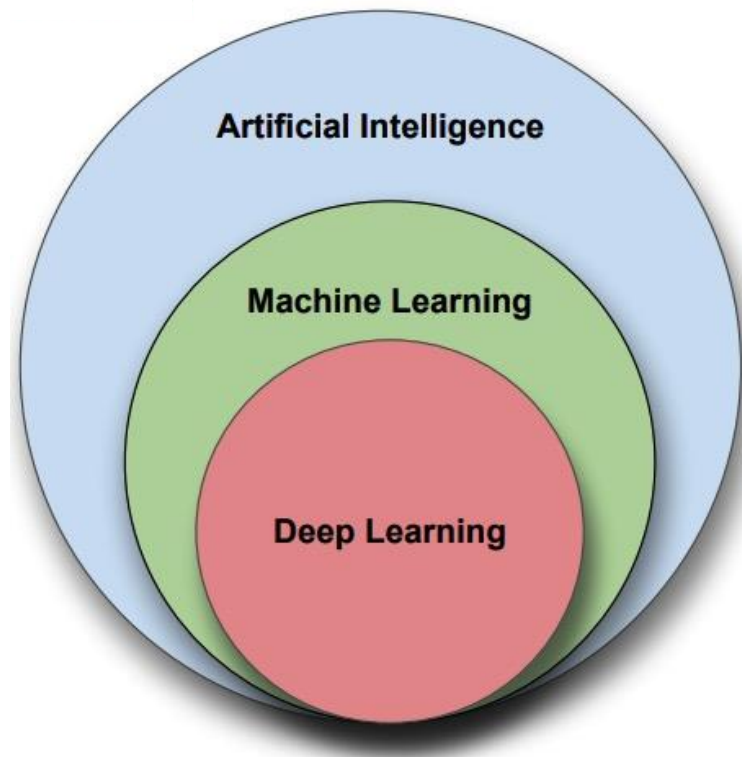
University
of Exeter

COMM113

Deep Learning

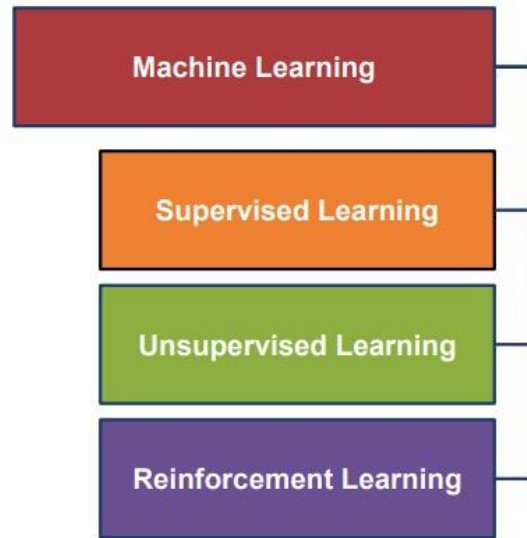
Relationship of AI, ML and DL

- **Artificial Intelligence (AI)** is anything about man-made intelligence exhibited by machines.
- **Machine Learning (ML)** is an approach to achieve AI.
- **Deep Learning (DL)** is one technique to implement ML.



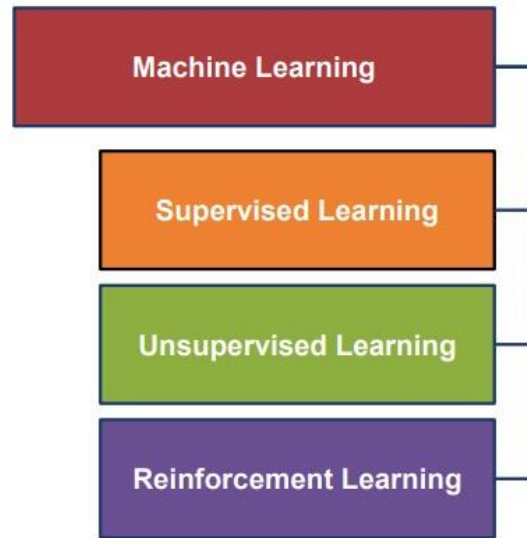
Types of ML Algorithms

- **Supervised Learning:** trained with labeled data; including regression and classification problems
- **Unsupervised Learning:** trained with unlabeled data; clustering and association rule learning problems.
- **Reinforcement Learning:** no training data; stochastic Markov decision process; robotics and self-driving cars.

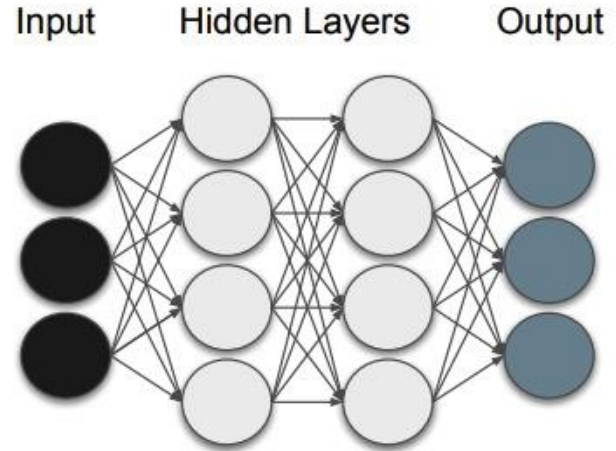
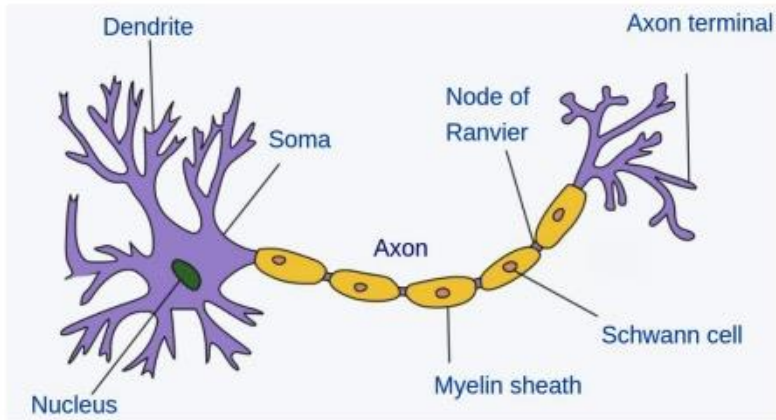


Types of ML Algorithms

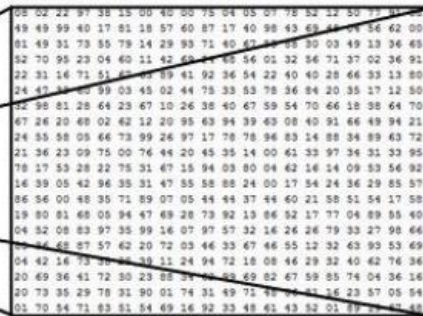
- “If I want to predict exam scores based on study hours, which type of ML is that?”
- “If Spotify recommends new songs based on your listening history, is that supervised, unsupervised, or reinforcement?”
- “What about training a drone to navigate a forest with no map?”



Artificial Neural Network



Inputs and Outputs



05	02	22	97	35	15	00	40	00	75	04	05	07	78	52	12	50	77	87	00
49	49	99	40	17	81	18	57	60	87	17	40	98	43	63	47	54	56	62	00
81	49	31	73	55	79	14	29	93	71	40	67	37	15	30	03	49	13	36	49
52	70	95	23	04	40	11	42	63	11	65	56	01	32	56	71	37	02	36	91
22	31	16	71	51	67	43	89	41	92	36	54	22	40	40	28	66	33	13	80
24	47	37	00	99	03	45	02	44	75	33	53	78	36	84	20	35	17	12	50
12	36	81	28	64	23	67	10	26	38	40	67	59	54	70	46	18	38	64	70
67	26	20	68	02	42	12	20	95	63	94	39	43	08	40	91	66	49	94	21
24	55	58	05	66	73	99	24	97	17	78	78	96	83	14	88	34	89	63	72
21	36	23	09	75	00	76	44	20	45	35	14	00	61	33	97	34	31	33	95
78	17	53	28	22	75	31	67	15	94	03	80	04	62	16	14	09	53	56	92
16	39	05	42	96	35	31	47	55	58	88	24	00	17	54	24	36	29	85	57
96	56	00	48	35	71	89	07	05	44	44	37	44	60	21	58	51	54	17	58
19	80	81	68	05	94	47	69	28	73	92	13	86	52	17	77	04	89	85	40
04	52	08	83	97	35	99	16	07	97	57	32	16	26	26	79	33	27	98	66
05	36	68	57	57	62	20	72	03	46	35	67	46	55	12	32	63	93	53	69
04	42	16	73	38	45	38	11	24	94	72	18	08	46	29	32	40	62	76	36
20	69	36	41	72	30	23	88	34	40	82	69	82	67	59	85	74	04	36	16
20	73	35	29	78	31	90	01	74	31	49	71	48	74	51	16	23	57	05	94
03	70	84	71	83	51	54	69	16	92	33	48	41	43	52	01	19	71	45	48

What the computer sees

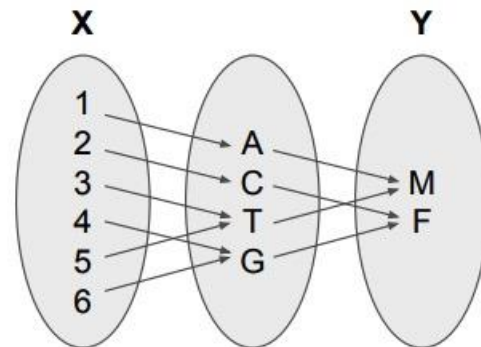
image classification

82% cat
15% dog
2% hat
1% mug

256 X 256
Matrix

DL model

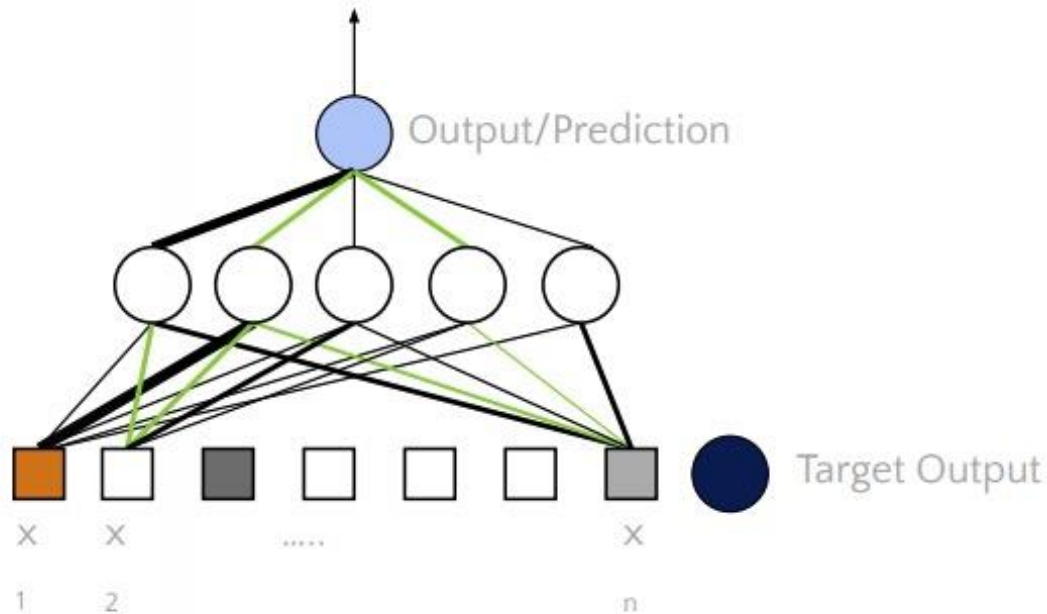
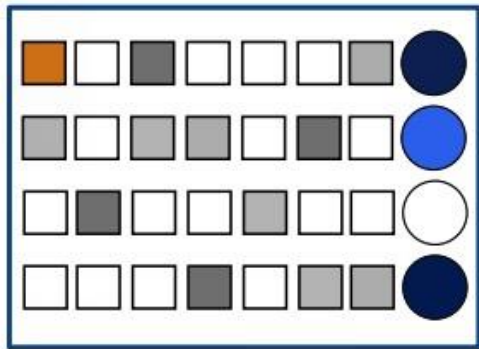
4-Element Vector



With deep learning, we are searching for a **surjective** (or **onto**) function f from a set X to a set Y .

Learning Principle

Dataset



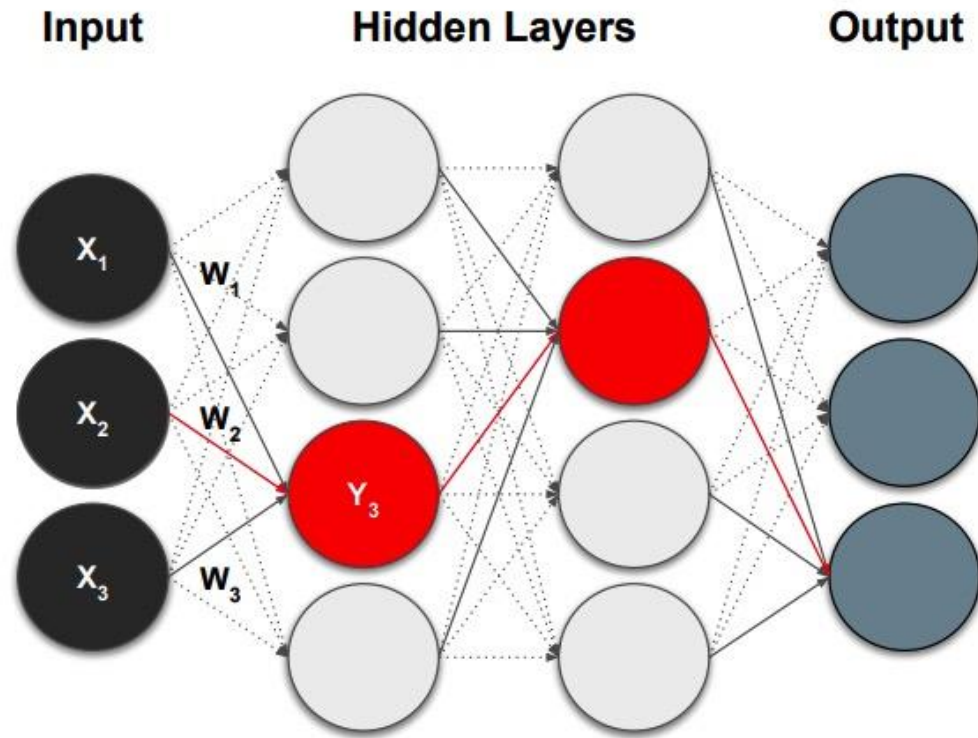
Error: $\text{Output/Prediction} - \text{Target Output} = 5$

Supervised DL with NN

From one layer to the next

$$Y_j = f\left(\sum_i W_i X_i + b_i\right)$$

f is the activation function,
 W_i is the weight, and b_i is the bias.



Training-Minimising the Loss

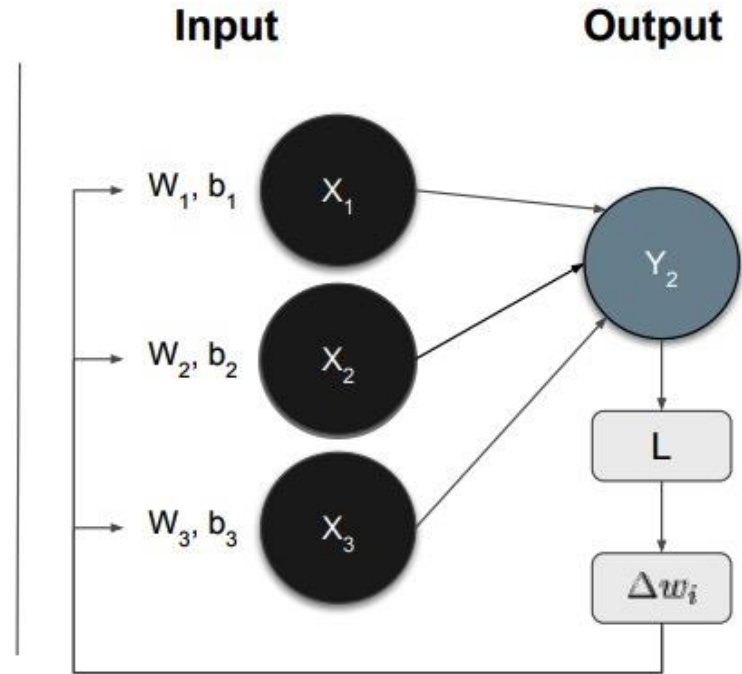
The loss function with regard to weights and biases can be defined as

$$L(\mathbf{w}, \mathbf{b}) = \frac{1}{2} \sum_i (\mathbf{Y}(\mathbf{X}, \mathbf{w}, \mathbf{b}) - \mathbf{Y}'(\mathbf{X}, \mathbf{w}, \mathbf{b}))^2$$

The weight update is computed by moving a step to the opposite direction of the cost gradient.

$$\Delta w_i = -\alpha \frac{\partial L}{\partial w_i}$$

Iterate until L stops decreasing.



Does anyone remember how linear regression minimises error? Difference?

What would happen if we never updated the weights after computing the loss?

Convolution Kernel

A convolution kernel slides over an image, extracting local features like edges. It's efficient because the same weights are reused across the image.

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Input

1	0	1
0	1	0
1	0	1

Filter / Kernel

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

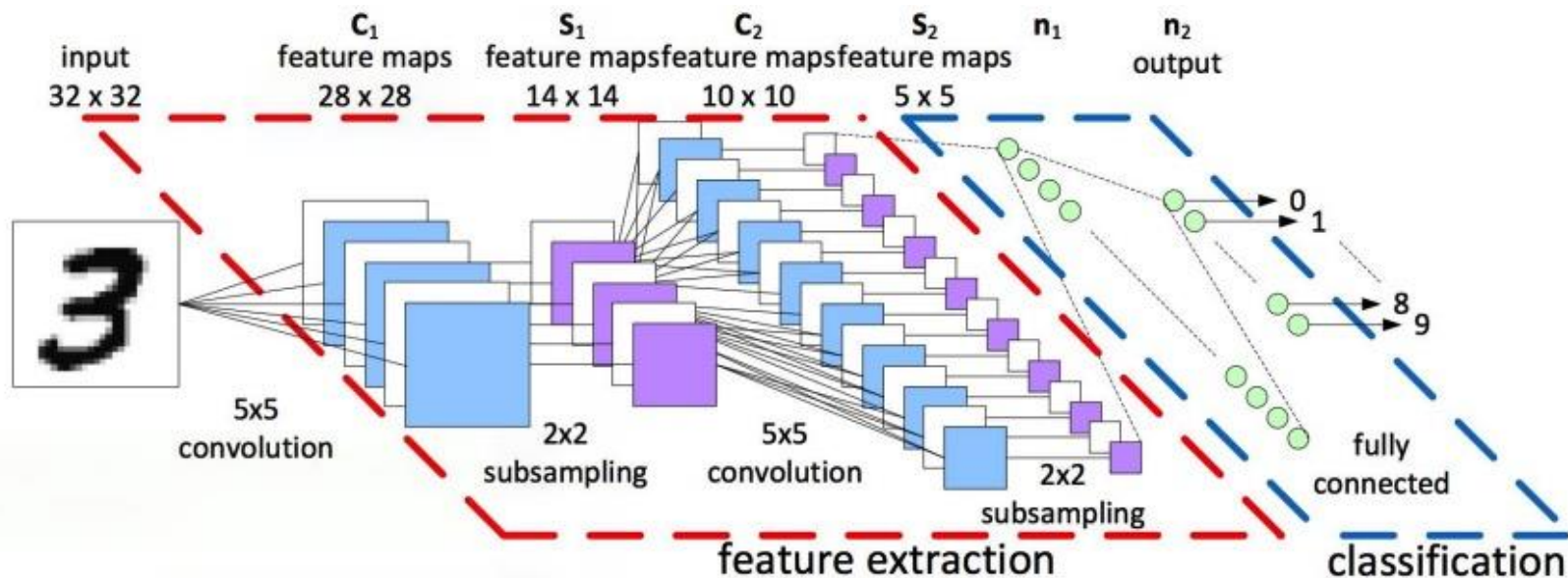
4		

[Convolution Kernel](#)

Why do you think sliding a small kernel over an image is better than fully connecting every pixel?

Convolutional Neural Networks

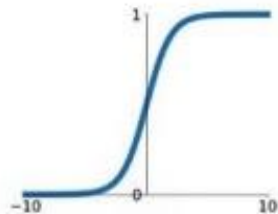
CNNs assume inputs are images and encode spatial structure. This reduces the number of parameters compared to fully connected layers, making them scalable and effective in vision.



Activation Functions

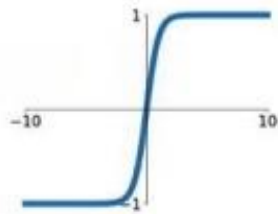
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



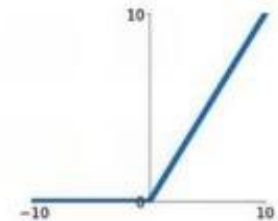
tanh

$$\tanh(x)$$



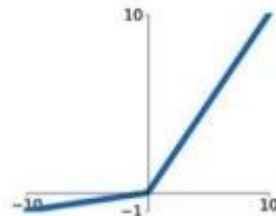
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

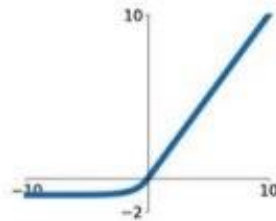


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

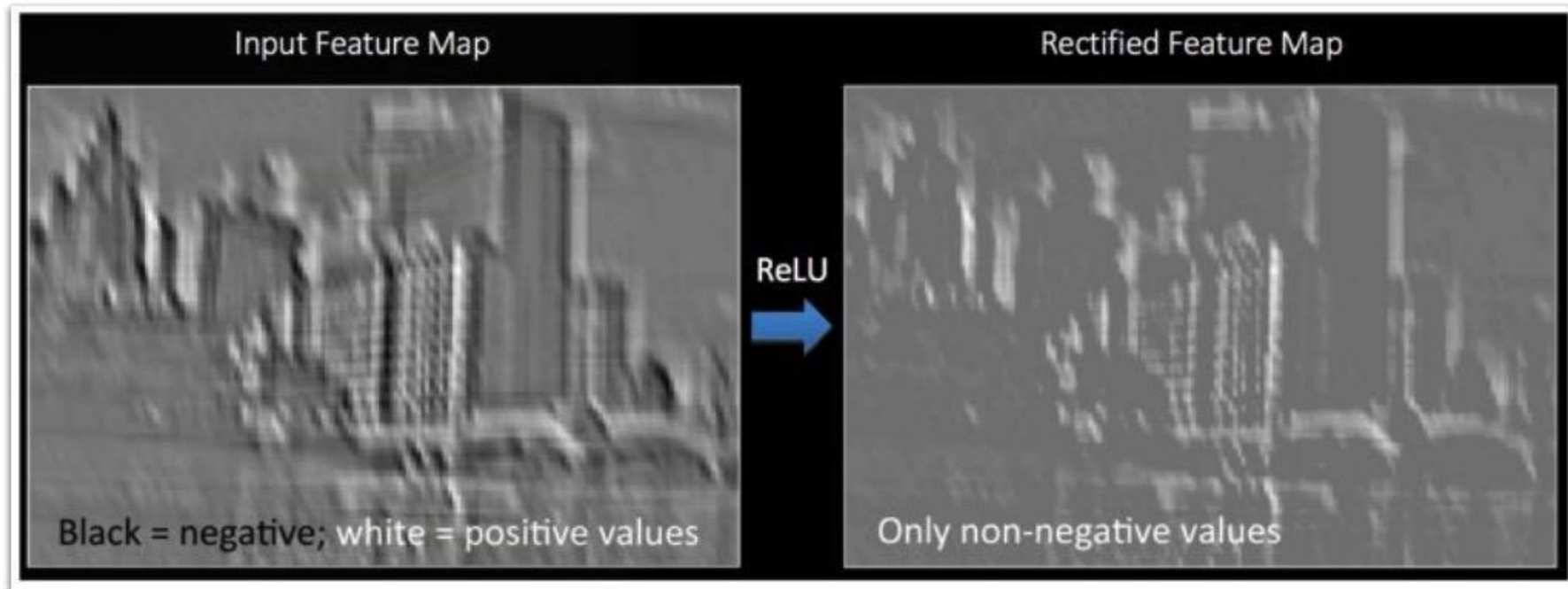
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



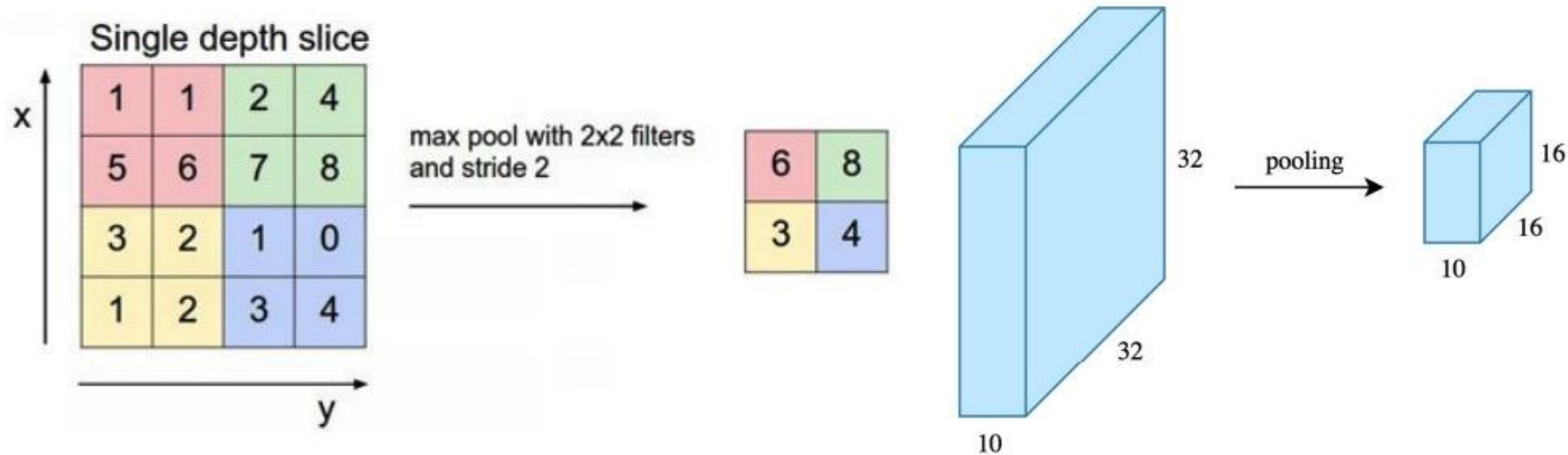
Activation functions introduce non-linearity. Without them, the whole network collapses into a simple linear function.

Non-Linearity (ReLU)



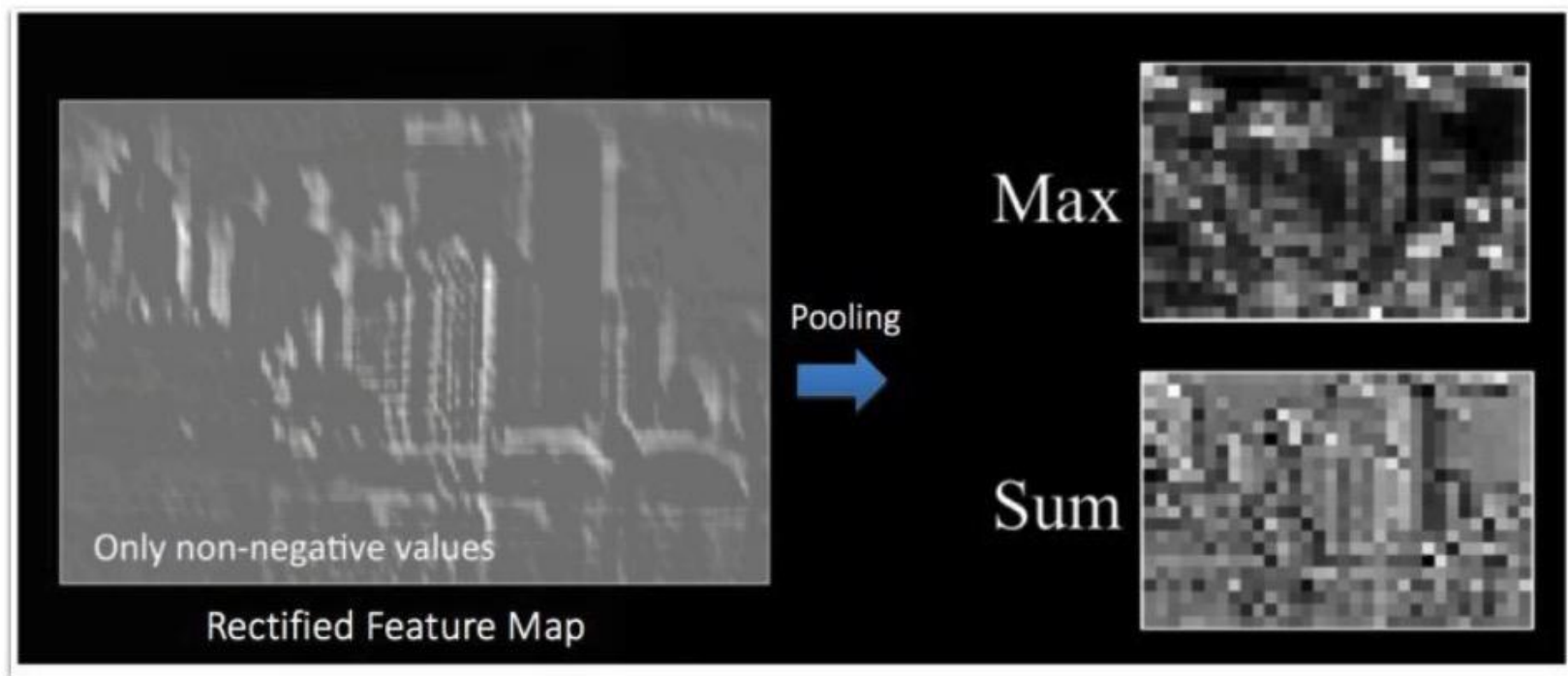
ReLU is the most widely used today. It's simple: zero for negatives, linear for positives. It avoids saturation and helps training converge faster.

Max Pooling



Pooling layers downsample the feature maps, reducing computational cost and adding invariance. Max pooling keeps the strongest activation in a region.

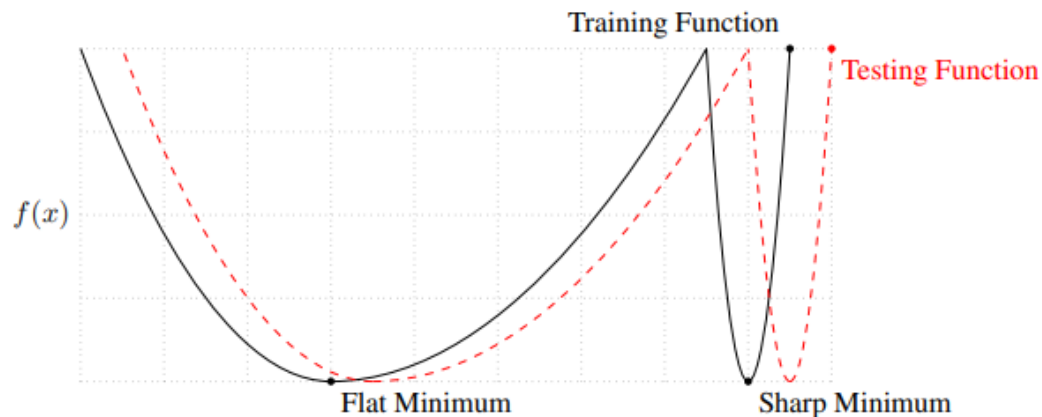
Max-Pooling; Sum-Pooling



Sum pooling averages or adds all activations. Max pooling picks the dominant one. Max is more common in vision tasks, but different problems may prefer different strategies.

(Deep Learning Methods for Vision | CVPR 2012 Tutorial)

Trustworthy: Generalisation



cardboard tusker



cement bell pepper



checkered platypus



straw bathtub



chocolate beaver



curly carton



dilapidated hog



colorful solar dish



cobblestone trailer



fluffy warplane



red badger



paper caldron



transparent cash machine



fabric police van



purple crayfish

[ML Generalisation](#)

Imagine training a self-driving car only on sunny-day images
— what happens if it sees snow?

Trustworthy: Robustness



x

“panda”

57.7% confidence

$+ .007 \times$



$\text{sign}(\nabla_x J(\theta, x, y))$

“nematode”

8.2% confidence

$=$



$x +$

$\epsilon \text{sign}(\nabla_x J(\theta, x, y))$

“gibbon”

99.3 % confidence

What could go wrong if a tiny change in an image makes a stop sign look like a speed-limit sign to the model?

[IBM Adversarial Robustness Toolbox](#)

What is PyTorch?

- Most popular research deep learning framework*
- Write fast deep learning code in Python (able to run on a GPU/many GPUs)
- Able to access many pre-built deep learning models (Torch Hub/torchvision.models)
- Whole stack: preprocess data, model data, deploy model in your application/cloud
- Originally designed and used in-house by Facebook/Meta (now opensource and used by companies such as Tesla, Microsoft, OpenAI)

Why Pytorch

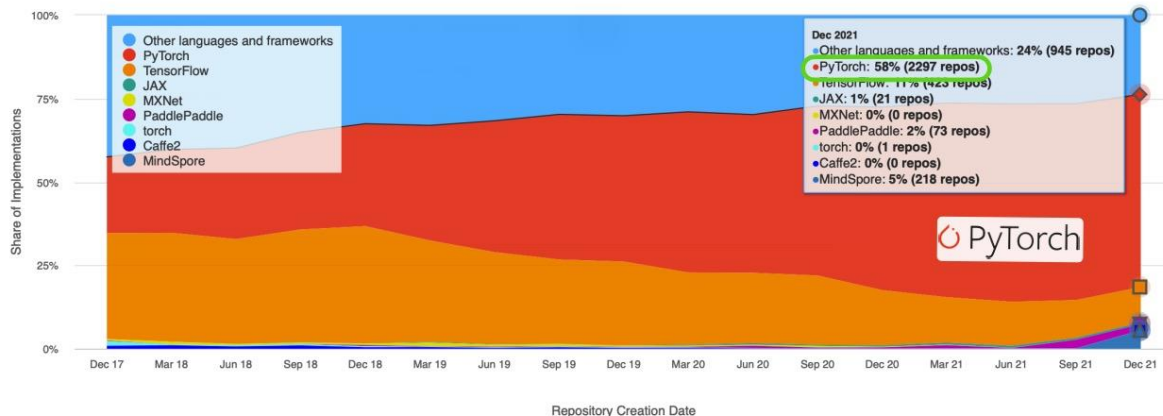


Trends

Quarter ▾ 2017-12-01 to 2021-12-31

Frameworks

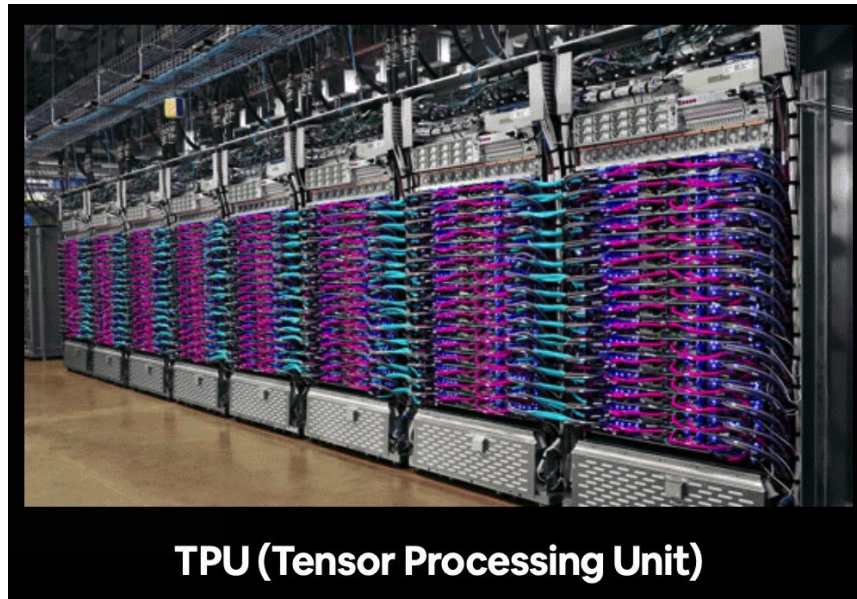
Paper Implementations grouped by framework



[Pytorch, Tensorflow, JAX](#)

PyTorch is preferred in academia for flexibility

What is a GPU/TPU?



Deep learning relies on GPUs and TPUs. These are processors specialised for matrix operations, massively speeding up training compared to CPUs.

Running Editor



If you don't have a powerful laptop, don't worry. Google Colab lets you run PyTorch code in the cloud with free GPU access. We'll use that in workshops.



Install

[Installing conda packages](#)



Here is an example command that installs NumPy version `2.2.2` and its dependencies:

```
conda install numpy=2.2.2
```



Downloading a package file

```
# Replace <PACKAGE> with the name of the package you want to install
# Optionally, specify the package <VERSION>
conda install <PACKAGE>=<VERSION> --download-only
```

[Install packages using “pip”](#)

Ensure you can run pip from the command line

Additionally, you'll need to make sure you have [pip](#) available. You can check this by running:

[Unix/macOS](#) [Windows](#)

```
python3 -m pip --version
```

[Unix/macOS](#) [Windows](#)

```
python3 -m pip install "SomeProject"
```

To install a specific version:

[Unix/macOS](#) [Windows](#)

```
python3 -m pip install "SomeProject==1.4"
```

If you want to run code locally, you can install libraries with pip or conda. Documentation is linked here. For most of you, Google Colab is easiest.

Python and Pytorch



COMM113.ipynb ☆

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all ▼



[2]

✓ 0s

```
import torch
import sys

print(sys.version)
print(torch.__version__)
```

```
3.12.11 (main, Jun 4 2025, 08:56:18) [GCC 11.4.0]
2.8.0+cu126
```

1

```
import torchvision
import torchvision.transforms as transforms

# Define transforms (convert to tensor and normalize)
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,)) # Normalize to [-1, 1]
])

# Load training dataset
train_dataset = torchvision.datasets.FashionMNIST(
    root='./data',          # Directory to save/load data
    train=True,             # Load training set
    download=True,          # Download if not already present
    transform=transform     # Apply transforms
)
```

```
100%|██████████| 26.4M/26.4M [00:01<00:00, 18.8MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 301kB/s]
100%|██████████| 4.42M/4.42M [00:00<00:00, 5.59MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 12.1MB/s]
```

Tensors

A tensor is a more general concept representing an n-dimensional array of numbers where 'n' is the tensor's rank or order. Scalars are 0-dimensional tensors, vectors are 1-dimensional, and matrices are 2-dimensional; higher-order tensors have three or more dimensions.

Attributes of a tensor 't':

- `t = torch.randn(1)`

`requires_grad`- making a trainable parameter

- By default False
- Turn on:
 - `t.requires_grad_()` or
 - `t = torch.randn(1, requires_grad=True)`
- Accessing tensor value:
 - `t.data`
- Accessing tensor gradient
 - `t.grad`

`grad_fn`- history of operations for autograd

- `t.grad_fn`

```
N, D = 3, 4
```

```
x = torch.rand((N, D), requires_grad=True)
y = torch.rand((N, D), requires_grad=True)
z = torch.rand((N, D), requires_grad=True)
```

```
a = x * y
b = a + z
c = torch.sum(b)
```

```
c.backward()
```

```
print(c.data)
print(x.data)
print(x.grad)
```

```
↩ tensor(9.3976)
  tensor([[0.9439, 0.9525, 0.0238, 0.4017],
          [0.6814, 0.6832, 0.4231, 0.1255],
          [0.0926, 0.4367, 0.3544, 0.9296]])
tensor([[0.5641, 0.8055, 0.8669, 0.3881],
        [0.0416, 0.1870, 0.3808, 0.9177],
        [0.4381, 0.2212, 0.9218, 0.1808]])
```

Gradient Descent

```
N, D = 3, 4
```

```
x = torch.rand((N, D), requires_grad=True)
y = torch.rand((N, D), requires_grad=True)
z = torch.rand((N, D), requires_grad=True)
```

```
a = x * y
b = a + z
c = torch.sum(b)
```

```
c.backward()
```

```
print(c.data)
print(x.data)
print(x.grad)
```

```
⇒ tensor(9.3976)
tensor([[0.9439, 0.9525, 0.0238, 0.4017],
        [0.6814, 0.6832, 0.4231, 0.1255],
        [0.0926, 0.4367, 0.3544, 0.9296]])
tensor([[0.5641, 0.8055, 0.8669, 0.3881],
        [0.0416, 0.1870, 0.3808, 0.9177],
        [0.4381, 0.2212, 0.9218, 0.1808]])
```

```
▶ x = x - 0.1 * x.grad
```

```
a = x * y
b = a + z
c = torch.sum(b)
```

```
print(c.data)
```

```
⇒ tensor(8.9959)
```

Dataset

FashionMNIST has 10 labels — how many outputs should our final layer have?

If we resized the images from 28×28 to 56×56 , what would that do to the input size?

```
import torchvision
import torchvision.transforms as transforms

# Define transforms (convert to tensor and normalize)
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,)) # Normalize to [-1, 1]
])

# Load training dataset
train_dataset = torchvision.datasets.FashionMNIST(
    root='./data',          # Directory to save/load data
    train=True,             # Load training set
    download=True,          # Download if not already present
    transform=transform     # Apply transforms
)
```

100%	██████████	26.4M/26.4M	[00:01<00:00, 18.8MB/s]
100%	██████████	29.5k/29.5k	[00:00<00:00, 301kB/s]
100%	██████████	4.42M/4.42M	[00:00<00:00, 5.59MB/s]
100%	██████████	5.15k/5.15k	[00:00<00:00, 12.1MB/s]

```
print(f"Training set size: {len(train_dataset)}")
print(f"Test set size: {len(test_dataset)}")
print(f"Total samples: {len(train_dataset) + len(test_dataset)}")

# Class names for FashionMNIST
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandals', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# Get a sample image and label
sample_image, sample_label = train_dataset[0]

print(f"\nExample data shape: {sample_image.shape}")
print(f"Example label: {sample_label} ({class_names[sample_label]})")
print(f"Pixel value range: [{sample_image.min():.2f}, {sample_image.max():.2f}]")
```

```
Training set size: 60000
Test set size: 10000
Total samples: 70000
```

```
Example data shape: torch.Size([1, 28, 28])
Example label: 9 (Ankle boot)
Pixel value range: [-1.00, 1.00]
```

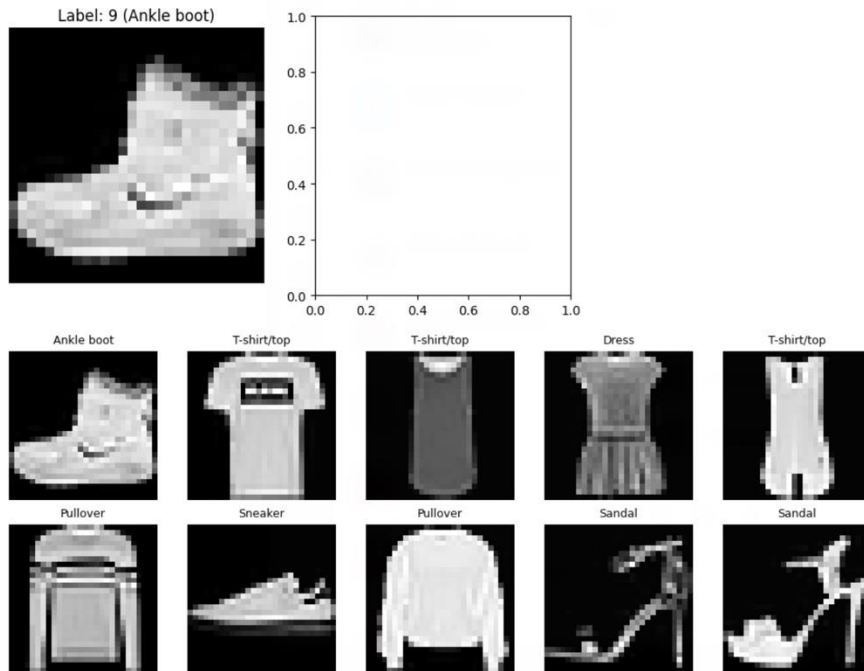
Dataset

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 4))

# Show normalized image
plt.subplot(1, 2, 1)
plt.imshow(sample_image.squeeze(), cmap='gray')
plt.title(f'Label: {sample_label} ({class_names[sample_label]})')
plt.axis('off')

# Show multiple examples
plt.subplot(1, 2, 2)
fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for i, ax in enumerate(axes.flat):
    image, label = train_dataset[i]
    ax.imshow(image.squeeze(), cmap='gray')
    ax.set_title(class_names[label], fontsize=9)
    ax.axis('off')
plt.tight_layout()
plt.show()
```



What's the input and output size of the MLP on FashionMNIST?

MLP model

```
# Define MLP model
class SimpleMLP(nn.Module):
    def __init__(self):
        super(SimpleMLP, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(28 * 28, 512)
        self.fc2 = nn.Linear(512, 256)
        self.fc3 = nn.Linear(256, 10)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.flatten(x)
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

Here's a simple multilayer perceptron: fully connected layers, activations, and softmax at the output for classification. We'll implement this in PyTorch.

Training

Device: Checks if GPU is available

Model: Creates an instance of the SimpleMLP model and moves it to the selected device (GPU/CPU)

Criterion: Defines the loss function (Cross Entropy Loss for multi-class classification)

Optimizer: Creates Adam optimizer with learning rate 0.001 to update model weights

Optimizer.zero_grad: Clears gradients from previous iteration (prevents accumulation)

Loss.backward: computes gradients

Optimizer.step: Updates model weights based on gradients

```
# Initialize model, loss, and optimizer
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = SimpleMLP().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Training function
def train_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    epoch_loss = running_loss / len(loader)
    epoch_acc = 100 * correct / total
    return epoch_loss, epoch_acc
```

Training

[11]
✓ 8m

```
# Training loop
num_epochs = 20
train_losses, train_accs = [], []
test_losses, test_accs = [], []

print(f"Training on {device}")
print("-" * 70)

for epoch in range(num_epochs):
    train_loss, train_acc = train_epoch(model, train_loader, criterion, optimizer, device)
    test_loss, test_acc = evaluate(model, test_loader, criterion, device)

    train_losses.append(train_loss)
    train_accs.append(train_acc)
    test_losses.append(test_loss)
    test_accs.append(test_acc)

    print(f"Epoch [{epoch+1}/{num_epochs}] | "
          f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.2f}% | "
          f"Test Loss: {test_loss:.4f} | Test Acc: {test_acc:.2f}%")

print("-" * 70)
print(f"Final Test Accuracy: {test_accs[-1]:.2f}%")
```



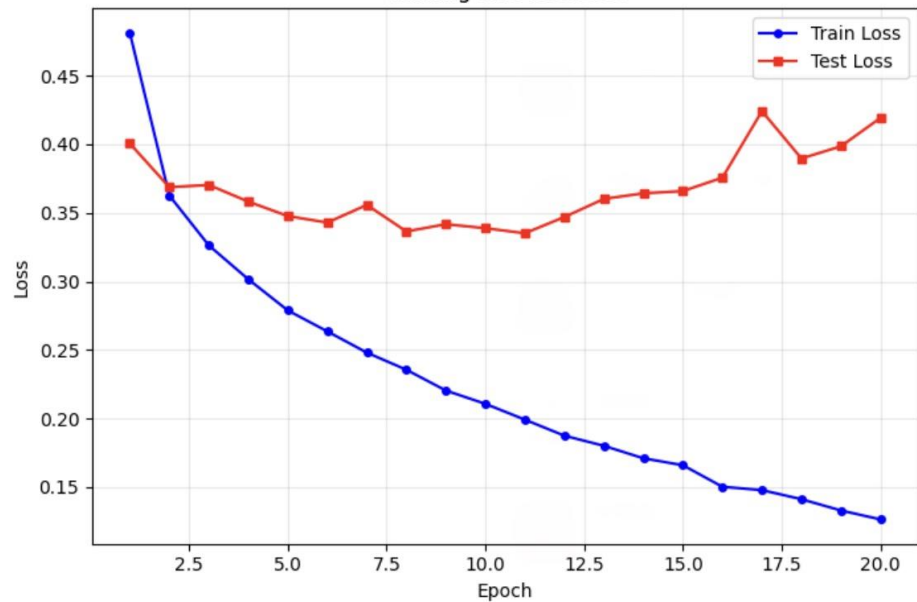
Training on cpu

Epoch [1/20]	Train Loss: 0.4814	Train Acc: 82.34%	Test Loss: 0.4007	Test Acc: 85.39%
Epoch [2/20]	Train Loss: 0.3627	Train Acc: 86.64%	Test Loss: 0.3687	Test Acc: 86.51%
Epoch [3/20]	Train Loss: 0.3263	Train Acc: 87.99%	Test Loss: 0.3704	Test Acc: 86.36%
Epoch [4/20]	Train Loss: 0.3016	Train Acc: 88.80%	Test Loss: 0.3582	Test Acc: 87.12%
Epoch [5/20]	Train Loss: 0.2788	Train Acc: 89.63%	Test Loss: 0.3477	Test Acc: 87.49%
Epoch [6/20]	Train Loss: 0.2636	Train Acc: 90.06%	Test Loss: 0.3429	Test Acc: 87.95%
Epoch [7/20]	Train Loss: 0.2481	Train Acc: 90.67%	Test Loss: 0.3558	Test Acc: 87.57%
Epoch [8/20]	Train Loss: 0.2356	Train Acc: 91.07%	Test Loss: 0.3365	Test Acc: 88.69%
Epoch [9/20]	Train Loss: 0.2204	Train Acc: 91.67%	Test Loss: 0.3417	Test Acc: 88.36%
Epoch [10/20]	Train Loss: 0.2107	Train Acc: 92.01%	Test Loss: 0.3388	Test Acc: 88.60%
Epoch [11/20]	Train Loss: 0.1992	Train Acc: 92.39%	Test Loss: 0.3351	Test Acc: 88.87%
Epoch [12/20]	Train Loss: 0.1875	Train Acc: 92.65%	Test Loss: 0.3470	Test Acc: 88.55%
Epoch [13/20]	Train Loss: 0.1801	Train Acc: 93.11%	Test Loss: 0.3602	Test Acc: 88.85%
Epoch [14/20]	Train Loss: 0.1710	Train Acc: 93.39%	Test Loss: 0.3643	Test Acc: 88.93%
Epoch [15/20]	Train Loss: 0.1659	Train Acc: 93.65%	Test Loss: 0.3659	Test Acc: 88.42%
Epoch [16/20]	Train Loss: 0.1502	Train Acc: 94.29%	Test Loss: 0.3757	Test Acc: 89.27%
Epoch [17/20]	Train Loss: 0.1478	Train Acc: 94.37%	Test Loss: 0.4242	Test Acc: 87.81%
Epoch [18/20]	Train Loss: 0.1412	Train Acc: 94.64%	Test Loss: 0.3897	Test Acc: 88.92%
Epoch [19/20]	Train Loss: 0.1329	Train Acc: 94.92%	Test Loss: 0.3986	Test Acc: 89.04%
Epoch [20/20]	Train Loss: 0.1264	Train Acc: 95.04%	Test Loss: 0.4193	Test Acc: 88.84%

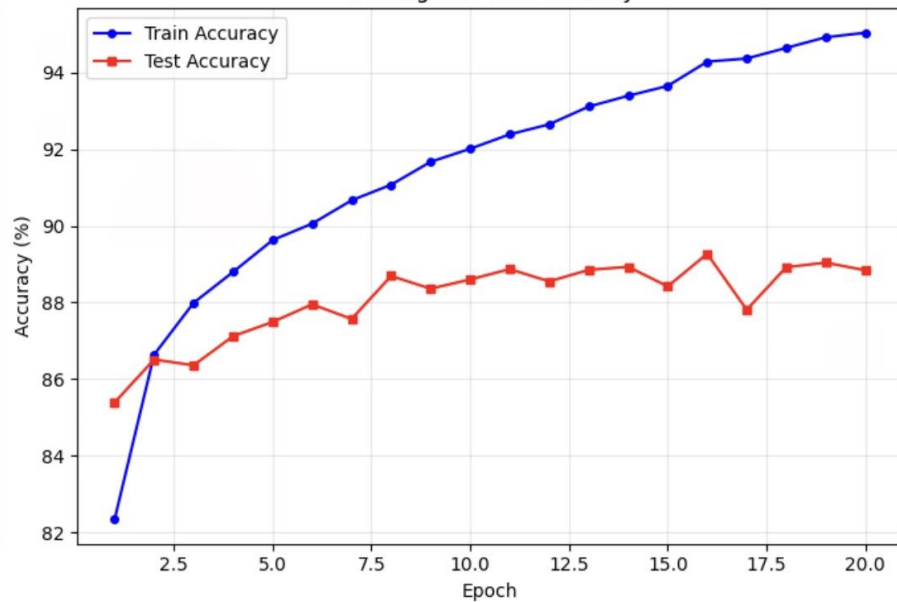
Final Test Accuracy: 88.84%

Training

Training and Test Loss



Training and Test Accuracy



Week 2 Workshop Task

Using Pytorch to develop a MLP on MNIST, with appropriate learning rate, optimizer, width, depth.

Train the model with 20 epochs, plot the training/test loss and training/test loss accuracy.

I will upload the task and the solution in the weekend.

Thanks